

Programowanie obiektowe

Wykład 8

Typy generyczne

Typy generyczne

Typy generyczne (ang. *generics*) to konstrukcje językowe umożliwiające pisanie kodu niezależnego od konkretnego typu danych, przy zachowaniu bezpieczeństwa typów w czasie kompilacji. Typy generyczne pozwalają na tworzenie klasy, metod, funkcji czy interfejsów działających na różnych typach danych, bez duplikowania kodu.

Korzyści

- **Reużywalność kodu** – nie trzeba definiować tej samej klasy osobno dla `int`, `string`, `double` itd.
- **Bezpieczeństwo typów** – kompilator pilnuje, czy używamy poprawnych typów
- **Czytelność i skalowalność** – lepsze API, unikamy użycia `object` i błędów rzutowania (`cast`)

Typy generyczne to narzędzie, które **rozszerza możliwości języków statycznie typowanych**, pozwalając pisać ogólny kod bez poświęcania bezpieczeństwa typów.

Typowanie statyczne

Typowanie statyczne (static typing) – typy zmiennych są określane **w czasie kompilacji**. Kompilator sprawdza poprawność typów i zgłasza błędy, zanim program zostanie uruchomiony.

Przykład (C#):

```
int number = 42;           // Typ znany w czasie kompilacji
string name = "Alice";
number = "text";           // Błąd kompilacji!
```

Zalety

- Błędy typów wychwytywane na etapie kompilacji
- Lepsza optymalizacja kodu (bo kompilator zna typy)
- Lepsze wsparcie IDE (podpowiedzi, refaktoryzacja)

Wady

- Mniej elastyczne
- Wymaga deklarowania typów (lub użycia inferencji)

Typowanie dynamiczne

Typowanie dynamiczne (*dynamic typing*) – typy zmiennych są określane **w czasie wykonania**. Program może się skompilować (lub zostać uruchomiony) bez informacji o typach.

Przykład (Python):

```
x = 10          # x to int
x = "hello"     # x teraz to string – brak błędu
print(x + 5)    # Błąd w czasie działania!
```

Zalety

- Większa elastyczność i szybkość pisania kodu
- Mniej "sztywności" w strukturze programów
- Przydatne w prototypowaniu i skryptach

Wady

- Błędy typów ujawniają się dopiero w czasie działania
- Mniejsze bezpieczeństwo typów
- Trudniejsze do refaktoryzacji

Inferencja typów

Wiele języków wspiera inferencję typu (np. `var` w Javie i C#, `auto` w C++).

Inferencja typów (ang. *type inference*) to mechanizm, w którym **kompilator** automatycznie określa typ zmiennej lub wyrażenia, **na podstawie kontekstu** – bez potrzeby jawnego podawania typu przez programistę.

Przykład (C# z `var`):

```
var name = "Alice";    // kompilator wie, że to string
var age = 30;          // kompilator wie, że to int
```

Mimo że nie podano typów, to **nie jest dynamiczne typowanie**: typy są nadal **statyczne** i **znane w kompilacji**, kompilator po prostu nie wymaga jawnego ich podania.

Zalety:

- Krótszy, czytelniejszy kod
- Mniej powtórzeń – typy nie są duplikowane (np. `Person person = new Person();`)
- Zachowane jest bezpieczeństwo statycznego typowania

Typowanie mieszane

Niektóre języki wspierają **zarówno statyczne, jak i dynamiczne typowanie** – pozwalają programiście wybrać, kiedy chce mieć typy znane w czasie kompilacji, a kiedy w czasie wykonania.

Przykład (C#)

```
// Typowanie statyczne
int number = 42;      // typ znany w czasie kompilacji

// Typowanie dynamiczne
dynamic x = "abc";
Console.WriteLine(x + 5); // "abc5"

x = 10;                // typ zmienia się w runtime
Console.WriteLine(x + 5); // 15
```

Daje wybór między elastycznością typowania dynamicznego a bezpieczeństwem typów statycznych.

- Inne przykłady:
 - Python (typowanie dynamiczne, ale z możliwością adnotacji typów),
 - TypeScript (nadzbiór JavaScript, dodający statyczne typowanie).

Typowanie silne vs słabe

To podział niezależny od statycznego/dynamicznego typowania

■ Typowanie silne (*strong typing*)

Język **nie pozwala** na niejawne (*implicit*) konwersje między niezgodnymi typami. Typy są **ściśle przestrzegane**.

Przykład: Python (dynamiczny, ale silnie typowany)

```
x = "5"
y = 2
print(x + y)    # TypeError
```

- Jasny, przewidywalny kod
- Bezpieczeństwo typów (trudniej popełnić błąd logiczny)
- Trochę bardziej uciążliwy w prostych przypadkach
- Wymaga więcej jawnych konwersji

■ Typowanie słabe (*weak typing*)

Język **pozwal**a na niejawne konwersje między różnymi typami. Typy można mieszać, a język stara się to interpretować.

Przykład: JavaScript (dynamiczny i słabo typowany)

```
console.log("5" + 1);    // "51"
console.log("5" - 1);    // 4
```

- Bardzo elastyczny
- Mniej kodu na konwersje
- Szybsze prototypowanie
- Potencjalnie nieprzewidywalny kod
- Błędy logiczne trudne do znalezienia

Problem: duplikacja kodu dla różnych typów

- Przykład: struktury operujące na różnych typach.

```
public class IntArray {  
    private int[] values;  
  
    public int Get(int idx)  
    {  
        return values[idx];  
    }  
  
    ...  
}
```

```
public class StringArray {  
    private string[] values;  
  
    public string Get(int idx)  
    {  
        return values[idx];  
    }  
  
    ...  
}
```

Powtarzamy ten sam kod dla różnych typów danych, np. `int`, `string`, `Customer`.

- Więcej kodu do napisania i utrzymania
- Trudniejsza refaktoryzacja (zmiany i poprawki należy nanosić jednocześnie w wielu miejscach)
- Duplikacja testów i dokumentacji
- Ryzyko niespójności (np. różne nazwy metod, komentarze)

Rozwiązanie – struktury uniwersalne

Rozwiązaniem jest stworzenie struktury uniwersalnej, np. opartej o typ `object`.

```
public class Array {  
    private object[] items;  
    private int count;  
  
    public void Add(object value)  
    {  
        items[count++] = value;  
    }  
  
    public object Get(int idx)  
    {  
        return items[idx];  
    }  
  
    ...  
}
```

Ale **rodzi to kolejne problemy.**

Uniwersalna struktura pozwala (dzięki rzutowaniu w górę) na przechowywanie danych dowolnego typu:

```
Array persons = new Array();  
persons.Add(new Person("Nowak"));
```

Ale **tracimy kontrolę typów:**

```
persons.Add("Kowalski");
```

Korzystanie z takiej struktury **wymaga jawnego rzutowania:**

```
Person person = (Person)persons.Get(0);
```

Co może prowadzić do **błędów runtime:**

```
person = (Person)persons.Get(1);  
// pod indeksem 1 znajduje się string
```

Problem: uniwersalne algorytmy

W językach statycznie typowanych dostosowanie algorytmu do operacji na różnych typach danych może wymagać duplikacji kodu:

```
void Sort(int[] array) { ... }  
void Sort(string[] array) { ... }  
void Sort(Employee[] array) { ... }
```

Rozwiązanie oferowane przez język C – funkcja `qsort` – opierało się na wskaźnikach typu `void*`:

```
void qsort(void* base, size_t nitems, size_t size,  
          int (*compar)(const void*, const void*));
```

Skorzystanie z niej wymagało podania własnej funkcji porównującej:

```
int compareInts(const void* a, const void* b) {  
    return (*(int*)a - *(int*)b);  
}  
  
/// main():  
int values[] = { 40, 10, 100, 90, 20 };  
qsort(values, 5, sizeof(int), compareInts);
```

Problem: brak kontroli typów, konieczność rzutowania wskaźników

Szablony klas - C++

- W języku **C++** szablon klasy definiujemy przy użyciu słowa kluczowego `template` :

```
template<typename T>
class Box {
public:
    T value;
};
```

- Można też użyć `template<class T>` - oba zapisy są równoważne.
- `T` jest **parametrem typu** - jest on nieznany w momencie definiowania szablonu, zostanie określony dopiero w miejscu jego użycia:

```
Box<int> intBox;
intBox.value = 42;

Box<std::string> strBox;
strBox.value = "hello";
```

- Nazwa `T` to tylko element konwencji (`T` - Type), inne spotykane:
 - `U`, `V` - kolejne parametry typu
 - `E` - Element
 - `TResult`, `TValue`, etc. - warto używać czytelnych skrótów, gdy typy mają konkretne znaczenie

Klasy generyczne w Javie

- To odpowiednik szablonów z C++.
- Parametry typu (`T`) podajemy po nazwie klasy.

```
public class Box<T> {  
    private T value;  
  
    public void set(T value) { this.value = value; }  
    public T get() { return value; }  
}
```

- Tworząc referencję klasy generycznej należy przekazać konkretny typ.
- Nie jest konieczne powtarzanie parametru przy tworzeniu obiektu.

```
Box<String> strBox = new Box<>();  
strBox.set("hello");
```

- Parametrem klasy generycznej może być dowolny typ obiektowy.
- Dla typów prostych należy użyć typów kopertowych (np. `int` → `Integer`, `double` → `Double`)

```
Box<Integer> intBox = new Box<>();  
intBox.set(42);
```

- *Boxing* (zamiana `int` na `Integer`) i *unboxing* (`Integer` na `int`) jest automatyczny.

Klasy generyczne w C#

- Składnia deklaracji taka sama jak w Javie (`<T>`).

```
public class Box<T>
{
    public T Value { get; set; }
}
```

- Tworząc obiekt typu generycznego należy przekazać parametr.

```
var strBox = new Box<string> { Value = "hello" };
```

- Może to być również typ prosty (prymityw: `int` , `double` , `bool`), a także inne typy wartościowe (`struct`).

```
var intBox = new Box<int> { Value = 42 };
```

- Kompilator zapewnia bezpieczeństwo typów i pełen dostęp do typu `T`

Konkretyzacja szablonu (C++)

- W języku C++ **konkretyzacja** szablonu następuje w momencie kompilacji.
- Kompilator tworzy na podstawie szablonu konkretną klasę dla **każdego użytego typu** (`Box<int>` , `Box<std::string>` itd.).

```
Box<int> intBox;  
Box<std::string> strBox;
```

- To w pełni niezależnie od siebie klasy - osobno kompilowane.

```
class Box {  
public:  
    int value;  
};
```

```
class Box {  
public:  
    std::string value;  
};
```

- Typ `T` jest w pełni znany i dostępny podczas kompilacji i runtime.
- Szablon klasy to nie klasa - nie można z niego tworzyć obiektów

```
Box box; // ❌ Error
```

- Klasa powstała przez konkretyzację szablonu „pilnuje” typów

```
intBox.value = "hello"; // ❌ Error
```

Realizacja klas generycznych w Javie

- Java **nie zachowuje informacji o typie generycznym w czasie wykonania** - *type erasure*.
- W trakcie kompilacji wszystkie wystąpienia `T` są zastępowane przez `Object`.
- Rzutowania są dodawane automatycznie przez kompilator.

```
Box<Integer> intBox = new Box<>();  
Box<String> strBox = new Box<>();
```

- W czasie wykonania **oba obiekty to** `Box`, bez informacji o typie:

```
System.out.println(intBox.getClass() == strBox.getClass()); // true
```

Efekty:

- Brak dostępu do `T` w runtime
- Nie można stworzyć obiektu typu `T` (`new T()`)
- Nie da się stworzyć tablicy typu generycznego (`T[]`) - zamiast tego używa się tablic `Object[]`
- Nie da się użyć `instanceof`
- Rzutowanie na typ `T` jest ryzykowne - wykonuje się bez sprawdzenia

Zalety: w pamięci jest tylko jedna definicja klasy `Box`

Implementacja generycznej kolekcji w Javie

- Z powodu *type erasure* nie możemy stworzyć tablicy typu `T[]`.
- Zamiast tego można użyć tablicy typu `Object[]`.

```
public class SimpleBox<T> {  
    private Object[] data;  
  
    public SimpleBox(int size) {  
        data = new Object[size]; // tworzenie tablicy Object dozwolone  
    }  
  
    public void set(int index, T value) {  
        data[index] = value;  
    }  
  
    @SuppressWarnings("unchecked")  
    public T get(int index) {  
        return (T) data[index]; // wymaga rzutowania  
    }  
}
```

- Wydobycie wartości wymaga rzutowania na typ `T` (niebezpieczne – kompilator nie sprawdzi typów w runtime).
- Adnotacja `@SuppressWarnings` zapobiega ostrzeżeniom kompilatora
- Lepszym rozwiązaniem jest użycie kolekcji z biblioteki standardowej (np. `ArrayList<T>`)

Realizacja generyków w C#

- Konkretyzacja następuje w *runtime* (JIT).
- C# zachowuje informacje o typach generycznych **w czasie wykonania** (*reification*).
- Typ `T` istnieje jako rzeczywisty typ – możliwy jest:
 - dostęp przez refleksję (`typeof(T)` , `GetType()`),
 - RTTI (`is`),
 - można rzutować na typ `T` ,
 - tworzyć obiekty typu `T` i tablice `T[]` .
- JIT generuje osobne wersje kodu **dla typów wartościowych** (`int` , `bool`), ale **współdzieli** kod dla typów referencyjnych.

```
Box<int> intBox = new Box<int>();  
Box<string> strBox = new Box<string>();  
  
Console.WriteLine(intBox.GetType()); // Box`1[System.Int32]  
Console.WriteLine(strBox.GetType()); // Box`1[System.String]
```

- Typy są różne i w pełni rozpoznawalne w czasie wykonania.

Parametry szablonów (C++)

- Poza parametrami typów (`typename T`), w C++ można też przekazać **wartość** jako parametr szablonu

```
template<typename T, int size>
class Array {
    T data[size];
};
```

- Rozmiar tablicy będzie ustalony w czasie kompilacji.

```
Array<int, 10> intArray; // tablica 10-elementowa typu int
```

- Parametry szablonów mogą być domyślne.

```
template<typename T = int, int size = 5>
class Array {
    T data[size];
};
```

- Pozwala to na konkretyzację z domyślnymi ustawieniami:

```
Array<> defaultArray;           // Array<int, 5>
Array<double> floatArray;       // Array<double, 5>
Array<char, 100> charArray;     // Array<char, 100>
```

- Java i C# nie wspierają parametrów wartościowych ani domyślnych.

Szablony funkcji w C++

- C++ pozwala na definiowanie szablonów funkcji/metod, przyjmujących parametry typu niezależnie od klasy, w której są zdefiniowane.

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

- Można jej użyć z **dowolnym typem** (jest on określany w miejscu wywołania).

```
int a = max(3, 7);           // T = int
double b = max(3.5, 2.1);    // T = double
```

- Kompilator **sam wnioskuję typ** z argumentów (tzw. *type deduction*)
- Można też jawnie podać typ:

```
double c = max<double>(3, 4);    // T = double
```

- Dla każdego użytego typu, kompilowana jest osobna wersja funkcji.

Szablony metod w C++

- Szablon metody może być zdefiniowany niezależnie od parametrów szablonu klasy.

```
template<typename T>
class Box {
    // szablon klasy sparametryzowany `T` może mieć szablon metody sparametryzowany `U`
    template<typename U>
    Box<U> convert() const {
        return Box<U>(static_cast<U>(value));
    }
};
```

- Może być też zdefiniowany w zwykłej klasie (nie-generycznej).

```
class Printer {
public:
    template<typename T>          // Parametr typu definiowany jest w metodzie a nie klasie
    void print(const T& value) {
        std::cout << value << std::endl;
    }
};
```

- Użycie jest analogiczne do przykładu z funkcją `max`.

```
Printer p;
p.print(42);           // int
p.print("Hello world"); // const char*
```

Generyczne metody w Javie

- Metoda generyczna deklaruje własny parametr typu (`<T>` przed zwracanym typem) niezależnie od parametrów typu klasy (jeśli w ogóle istnieją).
- Można stworzyć klasę niegeneryczną, zawierającą generyczne metody.

```
public class Utils {  
    public static <T> T getFirst(T[] array) {  
        if (array == null || array.length == 0) {  
            return null;  
        }  
        return array[0];  
    }  
}
```

- Kompilator sam może wywnioskować typ parametru generycznego przy wywołaniu metody:

```
String[] words = {"apple", "banana", "cherry"};  
  
String firstWord = Utils.getFirst(words);    // apple
```

- Parametr możemy przekazać jawnie przy wywołaniu: `Utils.<Integer>getFirst(...)`

Generyczne metody w C#

- W C# parametr typu (`<T>`) metody generycznej podaje się po jej nazwie.
- Tu również typ `T` metody generycznej jest niezależny od typu klasy, w której się znajduje

```
public T Identity<T>(T value)
{
    return value;
}
```

- Kompilator automatycznie wnioskuje typ `T` na podstawie argumentów wywołania lub możemy podać go jawnie: `Identity<int>(42)`

Przykład:

```
public static T? FirstOrDefault<T>(IEnumerable<T> a) {
    foreach (var item in a) {
        return item;
    }
    return default(T);
}
```

- `IEnumerable` to interfejs implementowany przez standardowe kolekcje (np. `List<T>`) i tablice (`T[]`).
- `default(T)` to domyślna wartość typu `T` (dla typów referencyjnych `null`).
- `T?` oznacza, że metoda `FirstOrDefault` może zwrócić `T` lub `null`

Specjalizacja szablonów w C++

To technika, dzięki której można **dostosować działanie szablonu** (klasy lub funkcji) dla **konkretnego typu lub kombinacji typów**.

- Nie zawsze ogólna definicja szablonu jest odpowiednia dla wszystkich typów

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

- Funkcja `max` działa na dowolnych typach, ale dla typu `char*` wyrażenie `a > b` porówna adresy, a nie zawartość tekstu.

```
cout << max("abc", "def") << endl; // "abc"
cout << max("def", "cab") << endl; // "def"
```

- Dlatego trzeba użyć np. `std::strcmp`, aby porównać treść łańcuchów znaków.

```
// Specjalizacja dla const char*
template<>
const char* max<const char*>(const char* a, const char* b) {
    return (std::strcmp(a, b) > 0) ? a : b;
}
```

- Specjalizacja szablonu to wyjątek od automatycznej konkretyzacji dla jednego, określonego typu
- W przypadku klas możliwa jest też częściowa specjalizacja (specjalizacja tylko niektórych parametrów)

Wymagania szablonów w C++

- Aby klasa utworzona z szablonu działała (i kompilowała się), typ którego używamy do skonkretyzowania szablonu musi spełniać określone wymagania.

```
template<typename T>
class Box {
    T value;

public:
    explicit Box(const T& initial) : value(initial) {}

    void set(const T& newValue) {
        value = newValue;
    }

    const T& get() const {
        return value;
    }
};
```

- `Box<T>` działa z każdym typem, który można kopiować i przypisać
- Jeśli parametrem szablonu jest klasa (`Box<Person>`), musi ona spełniać wymogi (np. konstruktor kopiujący, destruktor, operator podstawienia)
- To, czy typ spełnia wymagania, jest określone w momencie kompilacji skonkretyzowanego szablonu.

Ograniczenia typu w szablonach w C++

- Pisząc szablon, możemy założyć dowolny interfejs i liczyć że wszystko będzie działać - dostarczenie odpowiedniego typu należy do korzystającego z szablonu.
- Możemy np. wymagać określonej metody lub operatora

```
template<typename T>
class Box {
    ...
    bool insertIfGreater(const T& newValue) {
        if (newValue > value) {
            value = newValue;
            return true;
        }
        return false;
    }
};
```

- W nowoczesnym C++ można jawnie określić te wymogi przy pomocy `concept` lub `requires`, np.:

```
template<typename T>
requires std::totally_ordered<T> // wymaga operatorów <, <=, ==, ...
class Box {
    ...
};
```

Ograniczenie typu parametru w Javie

- W Javie możemy nałożyć ograniczenie na typ `T`, stosując składnię `<T extends NazwaKlasy>`.
- Przeważnie jest to stosowane, by wymóc, aby typ `T` implementował konkretny interfejs.

```
public class Box<T extends Comparable<T>> {  
    private T value;  
    ...  
}
```

- Wymagamy, aby przekazany typ `T` implementował `Comparable<T>` (to interfejs wbudowany w Javę).

```
public interface Comparable<T> {  
    int compareTo(T other);  
}
```

- Daje nam to pewność, że obiekty umieszczane w `Box` dają się porównywać.

```
public class Box<T extends Comparable<T>> {  
    ...  
    public boolean insertIfGreater(T newValue) {  
        if (newValue.compareTo(value) > 0) {  
            value = newValue;  
            return true;  
        }  
        return false;  
    }  
}
```

Ograniczenia na parametr typu w C#

- C# oferuje bardziej złożony system ograniczeń typów generycznych, oparty na składni `where T : ...`.

```
public class Box<T> where T : IComparable<T>
{
    ...
    public bool InsertIfGreater(T newValue) {
        if (newValue.CompareTo(Value) > 0)
        {
            Value = newValue;
            return true;
        }
        return false;
    }
}
```

- `where T : class` - `T` musi być typem referencyjnym
- `where T : struct` - `T` musi być typem wartościowym (np. `int`, `double`)
- `where T : new()` - `T` musi mieć publiczny konstruktor bezparametrowy
- `where T : BaseClass` - `T` musi dziedziczyć po określonej klasie
- `where T : InterfaceName` - `T` musi implementować dany interfejs
- Wielokrotne ograniczenia: `where T : BaseClass, Interface, new()`

Wymagania odnośnie operatorów (C#)

- Nie można bezpośrednio narzucić wymogu istnienia operatorów dla typu `T`, ale można to osiągnąć przez wymóg implementacji określonego interfejsu.

```
public static T Add<T>(T a, T b) where T : IAdditionOperators<T,T,T> {  
    return a + b;  
}
```

- `IAdditionOperators<TSelf, TOther, TResult>` to wbudowany interfejs z metodą `TResult operator +(TSelf left, TOther right)`.
- Metoda `Add` zadziała dla wszystkich typów które implementują `+`

```
Console.WriteLine(Add(3, 5));           // 8
```

- Aby `Add` działało dla naszych typów, musimy jawnie zaimplementować ten interfejs.

```
public class MyNumber : IAdditionOperators<MyNumber, MyNumber, MyNumber>  
{  
    ...  
    public static MyNumber operator +(MyNumber a, MyNumber b) {  
        return new MyNumber(a.Value + b.Value);  
    }  
}
```

Wildcards w Javie

- Wildcard to symbol `?`, który oznacza „dowolny nieznany typ”.
- Występuje głównie w sygnaturach argumentów typów generycznych (np. `List<?>`).
- Pozwala pisać uniwersalne metody, np. działające na kolekcjach generycznych sparametryzowanych dowolnym typem:

```
public static void printList(List<?> list) {  
    for (var n : list) {  
        System.out.println(n);  
    }  
}
```

- Metoda `printList(List<Object> list)` działałaby jedynie na liście obiektów.
- Metoda `printList(List<?> list)` wypisuje dowolną listę.

```
List<String> fruits = Arrays.asList("Apple", "Banana", "Cherry");  
printList(fruits);
```

- Z lity typu `List<?>` można wyjąć tylko `Object`, a włożyć tylko `null`.

Ograniczenie górne w Javie

Na wildcards można nakładać ograniczenia górne i dolne.

- Ograniczenie górne (`<? extends T>`) oznacza „dowolny typ **dziedziczący** ze wskazanego” (lub implementujący wskazany interfejs).
- Metoda `double sumOfList(List<Number> list)` zadziałałaby tylko dla listy `Number` , ale nie np. `Integer` (podtyp `Number`).

```
public static double sumOfList(List<? extends Number> list) {  
    double s = 0.0;  
    for (Number n : list)  
        s += n.doubleValue();  
    return s;  
}
```

- Ograniczenie górne pozwala przekazać listę dowolnego typu dziedziczącego z `Number` :

```
List<Integer> li = Arrays.asList(1, 2, 3);  
System.out.println("sum = " + sumOfList(li));
```

Ograniczenie dolne w Javie

Na wildcards można nakładać ograniczenia górne i dolne.

- Ograniczenie dolne (`<? super T>`) oznacza „dowolny typ **bazowy** względem wskazanego”.
- Ograniczenie **górne** (`<? extends T>`) zapewnia bezpieczną **produkcję**:
 - Z listy `List<? extends Number>` można bezpiecznie wyjąć element, będąc pewnym że będzie typu `Number` .
 - Wkładanie elementów nie jest bezpieczne – jeśli otrzymamy listę `List<Double>` , nie możemy tam włożyć wartości `Number` .
- Ograniczenie **dolne** (`<? super T>`) zapewnia bezpieczną **konsumpcję**:
 - Do listy `List<? super Integer>` można bezpiecznie wkładać wartości typu `Integer` – nawet jeśli otrzymamy np. listę typu `List<Object>` .
 - Wartości odczytujemy jedynie jako `Object` .

```
public static void addNumbers(List<? super Integer> list) {  
    for (int i = 1; i <= 10; i++) {  
        list.add(i);  
    }  
}
```

Ograniczenia dolne i górne w Javie

- W ramach jednej metody można stosować ograniczenie dolne i górne jednocześnie.

```
public class Util {  
  
    public static <T> void copy(List<? super T> destination, List<? extends T> source) {  
        for (T item : source) {  
            destination.add(item);  
        }  
    }  
}
```

- `List<? extends T> source` - możemy czytać z listy obiektów będących `T` lub podtypami `T`
- `List<? super T> destination` - możemy dodawać do listy obiekty `T` lub jego podtypy, bo lista może zawierać `T` lub nadtypy `T`

PECS: *"Producer extends, Consumer super"*

- `source` **produkuje** dane → `extends`
- `destination` **konsumuje** dane → `super`

Kowariancja i kontrawariancja w C#

- W C# odpowiednikiem ograniczeń górnych i dolnych z Javy są parametry typu `out` i `in`.
- Kowariancja** (`out`) deklaruje typ bezpieczny do odczytu, np.:

```
IEnumerable<out T>
```

- Sekwencja typu `string` może być traktowana jako sekwencja typu `object` (rzutowanie w górę).

```
IEnumerable<string> strings = new List<string>();  
IEnumerable<object> objects = strings; // OK – string jest subtypem object
```

- Kontrawariancja** (`in`) deklaruje typ bezpieczny do pisania (przekazania parametru), np.:

```
IComparer<in T>
```

- `IComparer` to interfejs zawierający metodę `int Compare(T x, T y)`, porównującą dwa elementy (np. przy sortowaniu).
- Komparator obiektów (`object`) może służyć do porównania napisów (`string`), ale nie na odwrót.

```
IComparer<object> objectComparer = ...;  
IComparer<string> stringComparer = objectComparer; // OK
```

- Kowariancja i kontrawariancja działa tylko na interfejsach i delegatach.

Klasy generyczne a dziedziczenie

Pomijając różnice składniowe, w C++, Javie i C# poniższe konstrukcje są dozwolone i logicznie spójne.

- Klasa generyczna dziedzicząca z typu niegenerycznego
 - Typ `T` dotyczy tylko `Derived`
 - Wzbogacamy istniejącą klasę o logikę opartą na typie

```
class Base { }  
class Derived<T> extends Base { }
```

- Klasa generyczna dziedzicząca z typu generycznego
 - Typ `T` jest przekazywany dalej do klasy bazowej
 - Możliwa jest specjalizacja (`extends Base<String>`)

```
class Base<T> { }  
class Derived<T> extends Base<T> { }
```

- Klasa niegeneryczna dziedzicząca z typu generycznego
 - Zamykamy parametr typu klasy bazowej
 - `T` staje się `String`

```
class Derived extends Base<String> { }
```

Złożone konstrukcje generyczne

- **Komponowanie typów** – typ generyczny może być parametrem innego typu generycznego.
- Przykład – lista list:

```
List<List<String>> stringMatrix = new ArrayList<>();  
stringMatrix.add(Arrays.asList("a", "b", "c"));  
stringMatrix.add(Arrays.asList("x", "y", "z"));
```

- Przykład – słownik list:

```
var studentGrades = new Dictionary<string, List<int>> {  
    ["Alice"] = new List<int> { 5, 4, 3 },  
    ["Bob"] = new List<int> { 4, 5 }  
};
```

- Typy generyczne z **wieloma parametrami**

```
public class Pair<A, B> {  
    public A First { get; }  
    public B Second { get; }  
  
    public Pair(A first, B second) {  
        First = first;  
        Second = second;  
    }  
}
```